

Kubernetes Autoscaling: HPA & VPA

A Comprehensive Guide for CKA Exam & Technical Interviews

Version 2.0 | Updated: May 2026 | For Latest Kubernetes Versions (1.28+)

Table of Contents

- Part 1: Horizontal Pod Autoscaler (HPA)
 - 1.1 Architecture & Core Concepts
 - 1.2 HPA Control Loop & Scaling Algorithm
 - 1.3 Metric Types & APIs (v2)
 - 1.4 Scaling Behavior (.spec.behavior)
 - 1.5 Advanced Scenarios & Edge Cases
 - 1.6 Practical Examples with Real-World Use Cases
 -
- Part 2: Vertical Pod Autoscaler (VPA)
 - 2.1 Architecture & Core Concepts
 - 2.2 VPA Modes (Auto, Recreate, Initial, Off)
 - 2.3 Resource Requests & Recommendation Engine
 - 2.4 VPA vs HPA: When to Use Each
 - 2.5 Advanced VPA Configuration
 - 2.6 Practical Examples
 -
- Part 3: HPA + VPA Combination
 - 3.1 Co-existing Strategies
 - 3.2 Anti-patterns & Gotchas
 -
- Part 4: Interview Q&A & CKA Exam Tips

PART 1: HORIZONTAL POD AUTOSCALER (HPA)

1.1 Architecture & Core Concepts

The Horizontal Pod Autoscaler automatically adjusts the number of pod replicas in a workload (Deployment, StatefulSet, or ReplicaSet) based on observed metrics such as CPU, memory, or custom application signals.

Key Principles

- Horizontal Scaling: Deploy more pods, not bigger pods
- Distributed Load: Spreads traffic across multiple pod instances
- Cost Optimization: Scale down unused resources during low demand
- High Availability: More pods = better fault tolerance

Horizontal vs. Vertical Scaling

Aspect	Horizontal (HPA)	Vertical (VPA)
Definition	Add/Remove pod replicas	Increase/Decrease CPU & Memory per pod
Approach	Scale OUT	Scale UP
Availability	High (multiple instances)	Lower (single larger pod)
Cost	Better for microservices	Better for monoliths
Deployment	Zero-downtime	Requires pod restart
Stateless Apps	Ideal	Not recommended
Databases	Not suitable	More appropriate

HPA Prerequisites

Metrics Server: Collects CPU/Memory metrics from kubelets (required for resource-based scaling)

API Version v2: Use autoscaling/v2 for advanced features (v1 is deprecated)

Resource Requests: Pods must have CPU/memory requests defined for percentage-based scaling

Custom Adapters: Prometheus Adapter for custom metrics, external adapters for external metrics

1.2 HPA Control Loop & Scaling Algorithm

The HPA operates as a Kubernetes controller running continuously within kube-controller-manager. It executes a deterministic control loop every 15 seconds by default.

The Control Loop: 5 Steps

1. Metric Polling: Query metrics.k8s.io API every 15s (default sync period)
2. Apply Tolerance: Check if metric deviation > 10% (default threshold)
3. Calculate Desired Replicas: Use ratio-based formula
4. Apply Stabilization: Look back at historical data to prevent flapping
5. Apply Scaling Policies: Enforce velocity limits (pods/percent per time window)

The Desired Replicas Formula

The core scaling algorithm:

```
[yaml]
desiredReplicas = ceil[currentReplicas ×
(currentMetricValue / targetMetricValue)]
```

6. Example: Current 5 replicas, target CPU 60%, observed CPU 90%:

```
[yaml]
Ratio = 90% / 60% = 1.5
Desired = ceil(5 × 1.5) = ceil(7.5) = 8 replicas
```

Metric Tolerance & Flapping Prevention

Without tolerance, small metric fluctuations near the target would constantly trigger scaling, destabilizing the control plane. Kubernetes applies a 10% default tolerance threshold.

```
[yaml]
Default Tolerance Range: [0.9 × target, 1.1 × target]
Example (target CPU 60%):
  - Lower bound: 54%
  - Upper bound: 66%
  - Action only if metric goes outside this band
```

Critical Note: Tolerance takes precedence over desired replicas calculation. Even if math suggests scaling, tolerance filters prevent action if metrics are within the acceptable range.

1.3 Metric Types & APIs (HPA v2)

HPA v2 supports four distinct metric types, each with different sourcing and configuration complexity.

Type	API Service	Use Case	Requires Adapter?	Complexity
Resource	metrics.k8s.io	CPU/Memory utilization	No (built-in)	Low
Custom (Pod)	custom.metrics.k8s.io	HTTP RPS, queue length per pod	Yes (Prometheus)	Medium
Custom (Object)	custom.metrics.k8s.io	Total requests to service	Yes (Prometheus)	Medium
External	external.metrics.k8s.io	AWS SQS, Kafka lag, db pool	Yes (specialized)	High

Resource Metrics (Type: Resource)

Most common metric type. Directly tied to pod resource requests/limits.

```
[yaml]
metrics:
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 70

- type: Resource
  resource:
    name: memory
    target:
      type: Utilization
      averageUtilization: 80
```

-  Pods must define resource requests for this to work:

```
[yaml]
resources:
  requests:
    cpu: 100m
    memory: 128Mi
```

```
limits:
  cpu: 500m
  memory: 512Mi
```

Custom Metrics (Type: Pods/Object)

Application-specific metrics like HTTP requests per second. Requires Prometheus + Prometheus Adapter.

```
[yaml]
# Per-Pod metric: Scale to maintain 100 RPS per pod
metrics:
- type: Pods
  pods:
    metric:
      name: http_requests_per_second
    target:
      type: AverageValue
      averageValue: "100"

# Per-Object metric: Total RPS across all pods
metrics:
- type: Object
  object:
    metric:
      name: requests_per_second
    describedObject:
      apiVersion: v1
      kind: Service
      name: my-service
    target:
      type: Value
      value: "30k"
```

External Metrics (Type: External)

Metrics originating outside Kubernetes cluster. Highly useful for proactive scaling based on upstream queue depths before the application experiences strain.

```
[yaml]
# Scale based on AWS SQS queue depth
metrics:
- type: External
  external:
    metric:
      name: sqs_queue_length
    selector:
      matchLabels:
        queue_name: orders-processing
    target:
      type: AverageValue
      averageValue: "100" # Maintain 100 messages per pod
```


1.4 Scaling Behavior (.spec.behavior)

HPA v2 introduced behavior tuning to control scaling velocity and prevent oscillation (flapping). Rules are applied AFTER the desired replicas calculation.

4.1 Stabilization Window (stabilizationWindowSeconds)

Prevents transient spikes from triggering unnecessary scaling. HPA maintains a history of desired replica counts within this window.

Scale-Down (Stable, Conservative): Default 300s (5 min). Selects HIGHEST desired replicas in window.

Scale-Up (Responsive, Aggressive): Optional, often 60s. Selects LOWEST desired replicas in window.

```
[yaml]
Example: 30s stabilization window shows desired replicas
[2, 3, 4, 3]
- Scale-up: picks min(2,3,4,3) = 2 (conservative, avoids
panic)
- Scale-down: picks max(2,3,4,3) = 4 (stable, prevents
premature scale-down)
```

4.2 Scaling Policies (Velocity Limits)

Control how fast scaling happens. Rate-limit pod additions/removals per time window.

Policy Type	Format	Effect
Pods	value: 4, periodSeconds: 60	Add/remove max 4 pods per 60s
Percent	value: 50, periodSeconds: 60	Add/remove max 50% of current replicas per 60s

```
[yaml]
behavior:
  scaleUp:
    policies:
      - type: Pods
        value: 4
        periodSeconds: 60
      - type: Percent
        value: 100
        periodSeconds: 60
    selectPolicy: Max # Pick policy allowing most change

  scaleDown:
    policies:
      - type: Pods
        value: 1
```

```
    periodSeconds: 60
  - type: Percent
    value: 50
    periodSeconds: 60
  selectPolicy: Min # Pick policy allowing least change
```

4.3 Policy Selection (selectPolicy)

When multiple policies exist, selectPolicy determines which applies:

Max (default): Uses policy allowing largest change (aggressive scale-up)

Min: Uses policy allowing smallest change (conservative scale-down)

Disabled: Prevents scaling in specified direction without changing metrics

1.5 Advanced Scenarios & Edge Cases

Scenario 1: Conflicting Metrics (CPU vs Memory)

When HPA monitors multiple metrics, it scales based on HIGHEST need:

```
[yaml]
metrics:
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 70
- type: Resource
  resource:
    name: memory
    target:
      type: Utilization
      averageUtilization: 80

Example:
CPU = 90% (target 70%) → suggests 5 replicas
Memory = 50% (target 80%) → suggests 2 replicas (no scale)
HPA scales to 5 (highest need wins)
```

Scenario 2: Multiple Custom Metrics

Same rule applies: HPA calculates desired replicas for each metric and uses the highest value.

```
[yaml]
metrics:
- type: Pods
  pods:
    metric:
      name: requests_per_second
    target:
      type: AverageValue
      averageValue: "1000"

- type: Pods
  pods:
    metric:
      name: db_connections
    target:
      type: AverageValue
      averageValue: "100"

Result = max(desiredReplicas_from_rps,
desiredReplicas_from_db)
```

Scenario 3: minReplicas vs maxReplicas Boundaries

Final replica count is bounded even if metrics and policies suggest otherwise:

```
[yaml]
finalReplicas = min(max(desiredReplicas, minReplicas),
maxReplicas)
```

Example:

minReplicas: 2

maxReplicas: 10

desiredReplicas (from metrics): 15

Policies allow: +5 pods per minute

Result: finalReplicas = min(max(15, 2), 10) = 10

1.6 Practical Examples with Real-World Use Cases

Example 1: E-Commerce Web App (Peak Traffic)

Requirements: Handle traffic spikes during sales, prevent scaling thrash

```
[yaml]
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: ecommerce-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 5
  maxReplicas: 100
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 75
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 30
      policies:
        - type: Percent
          value: 100 # Double pods every 30s if needed
          periodSeconds: 30
        - type: Pods
          value: 10
          periodSeconds: 30
      selectPolicy: Max
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 10 # Remove max 10% every 60s
          periodSeconds: 60
        - type: Pods
          value: 5
```

```
periodSeconds: 60
selectPolicy: Min
```

Example 2: API Server (Custom Metrics - RPS Based)

Scale based on requests-per-second using Prometheus Adapter

```
[yaml]
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-rps-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-server
  minReplicas: 3
  maxReplicas: 50
  metrics:
    - type: Pods
      pods:
        metric:
          name: http_requests_per_second
          target:
            type: AverageValue
            averageValue: "500" # 500 RPS per pod
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies:
        - type: Percent
          value: 50
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 25
          periodSeconds: 60
```

Example 3: Queue Consumer (External Metrics - SQS)

Proactive scaling based on message queue depth

```

[yaml]
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: queue-consumer-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: sqs-consumer
  minReplicas: 2
  maxReplicas: 80
  metrics:
  - type: External
    external:
      metric:
        name: sqs_approximate_number_of_messages_visible
        selector:
          matchLabels:
            queue_name: orders
      target:
        type: AverageValue
        averageValue: "100" # 100 messages per pod
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0 # Immediate response
to queue growth
    policies:
      - type: Percent
        value: 100 # Aggressive scale-up
        periodSeconds: 15
    scaleDown:
      stabilizationWindowSeconds: 600 # 10 min, queue must
be drained
    policies:
      - type: Percent
        value: 5
        periodSeconds: 60

```

PART 2: VERTICAL POD AUTOSCALER (VPA)

2.1 Architecture & Core Concepts

The Vertical Pod Autoscaler automatically adjusts CPU and memory requests/limits for pods based on historical usage data. VPA requires the metrics-server and maintains detailed metrics history to make informed recommendations.

Key Points

- Analyzes historical resource consumption over time (default: 8 days)
- Recommends optimal CPU/memory requests and limits
- Can automatically apply recommendations (requires pod restart)
- Uses statistical models to account for traffic patterns
- Prevents over-provisioning and under-provisioning

VPA Components

Recommender: Analyzes metrics, calculates recommendations

Updater: Applies changes by evicting/restarting pods

Admission Controller: Sets requests/limits for new pods

History DB: Stores metrics history for analysis

2.2 VPA Modes (Auto, Recreate, Initial, Off)

Mode	Behavior	Pod Restart?	Use Case
Auto	Update requests, evict pod if needed	Yes (if limits exceeded)	Production auto-tuning
Recreate	Same as Auto (pod restart guaranteed)	Yes	Aggressive optimization
Initial	Set requests only for new pods	No	Zero-downtime initial setup
Off	Calculate recommendations only	No	Dry-run / audit mode

Mode Descriptions

Auto Mode (Recommended)

```
[yaml]
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: my-app-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  updatePolicy:
    updateMode: "Auto" # Apply recommendations
                        automatically

# VPA will:
# 1. Update requests/limits for existing pods
# 2. Evict pods if new limits exceed current allocation
# 3. Recreate with new resource specs
```

Initial Mode (Safe for Existing Clusters)

Applies recommendations only to NEW pods, existing pods unaffected:

```
[yaml]
updatePolicy:
  updateMode: "Initial"

# Use case: Rolling out VPA in production without
# disrupting existing workloads
```

```
# Old pods keep current requests, new pods get optimized requests
```

Off Mode (Audit/Dry-run)

Only generates recommendations, no automatic changes:

```
[yaml]
updatePolicy:
  updateMode: "Off"

# VPA calculates recommendations, visible via: kubectl
describe vpa my-app-vpa
# Useful for: Testing, understanding current waste,
planning changes
```


2.3 Resource Requests & Recommendation Engine

How VPA Calculates Recommendations

VPA uses statistical analysis on historical metrics to recommend requests and limits:

```
[yaml]
Recommendation Logic:
1. Collect CPU/memory metrics over time (default: 8 days history)
2. Calculate percentiles: p50, p90, p99, p100 (max)
3. Request = p90 (90th percentile usage) * safety margin
4. Limit = min(p99, p100 * 1.5) (capped at max observed with safety margin)

Example:
If historical data shows:
- p50 (median): 100m CPU
- p90: 250m CPU
- p99: 400m CPU
- max observed: 450m CPU

VPA recommends:
- Request: 250m (to handle 90% of cases)
- Limit: 450m (allow occasional spikes)
```

Resource Targets & Policy

```
[yaml]
spec:
  resourcePolicy:
    containerPolicies:
      - containerName: app
        minAllowed:
          cpu: 50m
          memory: 64Mi
        maxAllowed:
          cpu: 2
          memory: 2Gi
        controlledResources: ["cpu", "memory"]
        controlledValues: RequestsAndLimits # or RequestsOnly

# Ensures recommendations stay within bounds
# minAllowed: Won't recommend below this
# maxAllowed: Won't recommend above this
```

2.4 VPA vs HPA: When to Use Each

Aspect	HPA	VPA
Scales	Number of replicas	CPU/Memory per pod
Response	Minutes	Slower (hours/days)
Pod Restart	No (most modes)	Yes (Auto/Recreate)
Stateless Apps	Perfect	Not ideal
Stateful Apps	Hard	Better
Database	Not suitable	Good
Cost	Lower (more pods)	Lower (right-sized)
Complexity	Low-Medium	Medium-High
Latency Impact	Better (scale out)	Worse (pod restart)

Decision Matrix

- Use HPA if:
 1. Workload is stateless (web servers, APIs)
 2. Need rapid response to traffic spikes
 3. Pod restart is unacceptable
 4. Distributed system with load balancing
- Use VPA if:
 5. Workload is stateful (databases, caches)
 6. Resource utilization is unpredictable
 7. Right-sizing is critical for cost
 8. Can tolerate pod restarts
- Use Both (HPA + VPA) if:
 9. High-traffic stateless workload (HPA for replicas, VPA for per-pod sizing)
 10. Variable per-pod demand AND traffic spikes

2.5 Advanced VPA Configuration

Example 1: Safe VPA Rollout (Initial Mode)

```
[yaml]
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: safe-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: production-app
  updatePolicy:
    updateMode: "Initial" # Affects new pods only
    minReplicas: 2
  resourcePolicy:
    containerPolicies:
      - containerName: app
        minAllowed:
          cpu: 100m
          memory: 128Mi
        maxAllowed:
          cpu: 1
          memory: 512Mi
        controlledResources: ["cpu", "memory"]
        controlledValues: RequestsAndLimits
```

Example 2: VPA for Database (Auto Mode, Conservative)

```
[yaml]
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: postgres-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: StatefulSet
    name: postgres
  updatePolicy:
    updateMode: "Auto"
    minReplicas: 1
  resourcePolicy:
    containerPolicies:
      - containerName: postgres
        minAllowed:
          cpu: 500m
          memory: 1Gi
        maxAllowed:
```

```
    cpu: 4
    memory: 8Gi
    controlledValues: RequestsOnly # Don't auto-set
limits
  recommendationUpdateMode: "Auto"
```

Example 3: VPA Audit Mode (Testing)

Deploy without making changes, review recommendations first:

```
[yaml]
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: audit-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: test-app
  updatePolicy:
    updateMode: "Off" # Dry-run mode

# Check recommendations:
# kubectl describe vpa audit-vpa
# kubectl get vpa audit-vpa -o yaml | grep recommendation
```

PART 3: HPA + VPA COMBINATION STRATEGIES

3.1 Co-existing Strategies

HPA and VPA can work together, but require careful configuration to avoid conflicts.

Recommended Combination Architecture

Use HPA for horizontal scaling (replicas) and VPA for vertical optimization (per-pod resources):

```
[yaml]
HPA Target: Maintain 70% CPU utilization across all pods
VPA Target: Right-size CPU/memory requests per pod

Workflow:
1. HPA adds/removes pods based on aggregate CPU
2. VPA analyzes individual pod usage, adjusts requests
3. HPA then re-evaluates based on optimized requests

Result: Optimal cluster utilization and cost
```

Full Example: Web App + VPA + HPA

```
[yaml]
---
# Deployment with no explicit resource requests (VPA will
# set)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3 # Will be scaled by HPA
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: app
          image: web-app:latest
          # NO resource requests/limits initially - VPA will
          # manage
---
# VPA: Right-size per-pod resources
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
```

```

    name: web-app-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  updatePolicy:
    updateMode: "Initial" # Start with new pods only
  resourcePolicy:
    containerPolicies:
      - containerName: app
        minAllowed:
          cpu: 100m
          memory: 128Mi
        maxAllowed:
          cpu: 1
          memory: 512Mi
---
# HPA: Scale replicas based on CPU across all pods
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 3
  maxReplicas: 50
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies:
        - type: Percent
          value: 50
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent

```

```
value: 10  
periodSeconds: 60
```

3.2 Anti-patterns & Gotchas

- ✗ **VPA + HPA on Same Metric:** Both trying to scale CPU → conflicts. Use HPA for replicas, VPA for per-pod tuning.
- ✗ **VPA Initial Mode with Rolling Updates:** Old pods keep stale requests. Pair with RollingUpdate strategy.
- ✗ **No minAllowed/maxAllowed:** VPA might recommend unreasonable values. Always set bounds.
- ✗ **VPA Off Mode in Prod:** Recommendations calculated but ignored. Follow up with deployments.
- ✗ **Insufficient Metrics History:** VPA waits 7-8 days for baseline. Only safe after warm-up period.

PART 4: INTERVIEW Q&A & CKA EXAM TIPS

Common Interview Questions

7. Q: How does the default 15-second HPA sync period affect scaling responsiveness?

A: The 15-second sync period is the absolute minimum reaction time. Even if you set `stabilizationWindowSeconds` to 5s, HPA only checks metrics every 15s (configured globally on controller-manager, not overridable in managed Kubernetes). This means:

- Scale-up can be delayed up to 15 seconds before action
- Metrics are only evaluated on sync cycles
- High-frequency metric collection (`scrape_interval < 15s`) doesn't help HPA directly

For latency-sensitive workloads, compensate by:

- Setting `minReplicas` higher
- Using external metrics for proactive scaling
- Configuring aggressive scale-up policies

8. Q: What's the difference between `.spec.behavior.stabilizationWindowSeconds` and `periodSeconds`?

A: `stabilizationWindowSeconds`: Historical lookback window to prevent flapping

- Examines past desired replica calculations within this window
- For scale-down, selects max to delay premature removal
- For scale-up, selects min to avoid panic scaling

`periodSeconds`: Time window for policy enforcement

- Rate-limits how fast scaling can happen
- Example: 4 pods max per 60 seconds means max 4 pod additions per minute

Analogy:

- `stabilizationWindowSeconds` = "Look back to confirm trend"
- `periodSeconds` = "Enforce rate limits per time window"

9. Q: How does HPA decide between multiple metrics?

A: HPA calculates desired replicas independently for each metric, then uses the HIGHEST value.

Example:

- CPU metric suggests 10 replicas
- Memory metric suggests 5 replicas

- Custom RPS metric suggests 8 replicas

Final decision: 10 replicas (highest need wins)

This ensures no single resource becomes a bottleneck. The tradeoff: over-provisioning on other metrics to prevent CPU starvation.

10. Q: When should I use VPA instead of HPA?

A: Use VPA (Vertical) when:

1. Workload is stateful (databases, stateful services)
2. Replicas are fixed or undesirable
3. Per-pod resource needs vary significantly
4. Cost optimization from right-sizing is critical
5. Pod restarts are acceptable

Avoid VPA when:

- Workload is stateless (use HPA)
- Pod restart causes unacceptable downtime
- Real-time responsiveness required (VPA is slower)

Use both when:

- Stateless service with variable per-pod demand
- Need both replica scaling (HPA) + per-pod tuning (VPA)

11. Q: What happens if HPA desired replicas exceeds maxReplicas?

A: HPA respects maxReplicas as hard boundary:

$$\text{finalReplicas} = \min(\max(\text{desiredReplicas}, \text{minReplicas}), \text{maxReplicas})$$

Example:

- Desired (from metrics): 150 replicas
- maxReplicas: 100
- Result: 100 replicas (capped)

The high load persists but can't scale further. Strategies:

1. Increase maxReplicas if cost permits
2. Implement request-shedding/queue-dropping
3. Use external metrics for proactive capacity planning

CKA Exam Tips

✓ **Know the Formula:** $\text{desiredReplicas} = \text{ceil}[\text{current} \times (\text{current_metric} / \text{target_metric})]$.
Be able to calculate by hand.

✓ **Understand Tolerance:** 10% default band $[0.9 \times \text{target}, 1.1 \times \text{target}]$. Scaling only outside this range.

✓ **Metric Precedence:** Multiple metrics: use highest desired replicas value. Know why this prevents bottlenecks.

✓ **API Versions:** Use autoscaling/v2, not v1. v1 missing many features (behaviors, multiple metrics).

✓ **Prerequisites:** Metrics Server required. Pods need resource requests. Know which adapters for which metric types.

✓ **Behavior Tuning:** `stabilizationWindowSeconds` for history, `periodSeconds` for rate-limiting. Be able to configure both.

✓ **VPA Modes:** Auto/Recreate apply changes, Initial affects new only, Off is dry-run.

✓ **Practice YAML:** Be comfortable writing full HPA specs. Know all metric types.

Common CKA Questions

- Write HPA spec that scales 5-50 pods, targets 70% CPU with aggressive scale-up (50%) and conservative scale-down (10%)
- Debug: HPA exists but pod count not changing despite high CPU. What could be wrong?
- Configure VPA for stateful set with rolling updates. How do you prevent downtime?
- Write spec combining HPA (scale replicas) + VPA (per-pod tuning). Explain interaction.
- Explain why tolerance threshold prevents scaling thrash. Give example numbers.
- Your HPA is scaling too slowly. What 3 things would you check/change?

Debugging HPA Issues

1. Check HPA Status: `kubectl get hpa`, `kubectl describe hpa <name>`. Look for "Metrics" row and "Current / Target"

1. Verify Metrics Server: `kubectl get deployment metrics-server -n kube-system`. Must exist and be running.

1. Check Pod Requests: `kubectl describe pod`. Every pod must have resource requests for metric-based scaling.

1. Examine Events: `kubectl describe hpa` shows recent scaling events. Look for warnings/errors.

1. Verify Metrics API: `kubectl get --raw /apis/metrics.k8s.io/v1beta1/nodes`. Should return metrics.

1. Check Tolerance: Even if CPU is 75% and target is 70%, may not scale if within 10% tolerance band.

1. Review Stabilization: Scale-down may be delayed by 300s (default). Check `.spec.behavior` settings.

Key Commands for CKA

`kubectl get hpa` — List all HPAs in namespace

`kubectl describe hpa <name>` — Show detailed HPA status, current/target metrics, events

`kubectl get vpa` — List all VPAs

`kubectl describe vpa <name>` — Show VPA status and recommendations

`kubectl get --raw /apis/metrics.k8s.io/v1beta1/nodes` — Verify metrics server is working

`kubectl edit hpa <name>` — Edit HPA spec in-place

`kubectl delete hpa <name>` — Remove HPA (pods will stay at current count)

`kubectl get deployment metrics-server -n kube-system` — Verify metrics-server deployment

Summary Checklist

- Before deploying HPA to production:
 11. Metrics Server running and healthy
 12. All pods have resource requests defined
 13. API version is autoscaling/v2
 14. Stabilization windows configured (60s scale-up, 300s scale-down)
 15. Scaling policies tested in lower env
 16. minReplicas and maxReplicas set appropriately
 17. Monitoring in place for HPA events
 18. Load testing performed to validate behavior
- For VPA:
 19. Metrics Server running (same as HPA)
 20. Understanding: VPA takes 7-8 days to generate recommendations
 21. Test in "Off" mode first to review recommendations
 22. Set minAllowed/maxAllowed resource bounds
 23. Choose appropriate updateMode (Initial for safe rollout)
 24. Plan for pod restarts (Auto/Recreate modes)
 25. Monitor VPA controller logs

Conclusion

Kubernetes autoscaling is essential for cloud-native applications. HPA handles rapid scaling based on real-time demand, while VPA optimizes resource efficiency over time. Mastery requires understanding the underlying algorithms, edge cases, and interaction with your metrics infrastructure.

- Key Takeaways:
 26. HPA: Scale OUT (replicas) in response to demand
 27. VPA: Scale UP (per-pod resources) for efficiency
 28. Both together: Optimal cluster utilization
 29. Know the formula, understand tolerance, practice YAML
 30. Metrics Server is mandatory for both
 31. Test extensively before production deployment